NexusHR

AI-Enabled Enterprise HR & Workforce Intelligence Platform

Production-grade Java full-stack HR system with employee lifecycle management, attendance, payroll, performance reviews, JWT security, PostgreSQL persistence, real-time operations readiness, and AI-style workforce insights.

Prepared By	Daksh Kumar
Domain	Zidio Development - Java Full Stack
Date	June 2026
Version	1.0 Submission Report
Repository Folder	/Users/dakshkumar/Desktop/NexusHR

Employee Lifecycle • Attendance • Payroll • Performance • AI Insights • Notifications • Monitoring

1. Project Overview

NexusHR is designed as a modern HR and workforce intelligence platform for mid-to-large enterprises. The platform reduces manual HR operations by centralizing employee records, attendance, leave decisions, payroll execution, performance reviews, notifications, and audit trails in one secured full-stack system.

The project follows the Zidio Java Full Stack brief by using Spring Boot for backend services, React with TypeScript for the frontend operations console, relational persistence through PostgreSQL-ready JPA entities, JWT authentication, Dockerized services, Kubernetes manifests, and monitoring support through Actuator, Prometheus, and Grafana.

Target Users

User Type	Primary Needs
HR Admin	Manage employee lifecycle, payroll, audit logs, and operational reporting.
Manager	Review team data, approve or reject leave, inspect performance and engagement.
Employee	Self-service login foundation for profile, payslip, leave, and notification access.
Leadership	High-level workforce metrics, attrition risk, attendance health, and payroll readiness.

Business Value

The system improves HR efficiency by replacing spreadsheet-driven workflows with API-backed workflows and role-aware dashboards. It provides operational visibility into workforce size, attrition risk, payroll status, attendance health, and pending approvals.

2. Functional Requirements Coverage

ID	Feature	Implementation Status	Acceptance Evidence
F-01	Employee Lifecycle Management	Implemented	JPA employee model, create/update/offboard endpoints, department mapp
F-02	Attendance & Leave Management	Implemented	Attendance records, leave request list, approval/rejection workflow, notifica
F-03	Payroll Processing	Implemented	Payroll run endpoint calculates gross, tax, benefits, net salary, and payslip
F-04	Performance Management	Implemented	Performance review model with manager score, peer score, goal completi
F-05	AI Workforce Insights	Partially Implemented	Heuristic attrition and skill gap cards; Spring AI provider integration remain
F-06	Admin & Manager Dashboards	Implemented	React operations console with JWT login, role display, metrics, and workfl
F-07	Notification & Communication	Partially Implemented	Notification records generated for payroll/leave; mail/WebSocket depende

The implementation focuses on runnable local functionality plus production-ready scaffolding. Features that require third-party credentials, such as SMS delivery and live AI model calls, are represented by clean integration points and documented extension paths.

3. Technology Stack

Layer	Technology	Purpose
Backend	Java 17-compatible Spring Boot 3.3	REST APIs, validation, business workflows, security.
Frontend	React + TypeScript + Vite	Fast operational console with typed API integration.
Database	PostgreSQL 17 in Docker, H2 for local dev	ACID persistence and zero-setup development path.
Persistence	Spring Data JPA + Hibernate	Entity modelling and repository abstraction.
Migrations	Flyway	Versioned database schema management.
Security	Spring Security + JWT + Argon2	Role-based authorization and secure password hashing.
Cache	Redis 7	Session/cache readiness in Docker profile.
Monitoring	Actuator + Prometheus + Grafana	Health checks, metrics, and dashboard provisioning.
Deployment	Docker, Docker Compose, Kubernetes HPA	Containerized local and cloud-ready deployment.
CI/CD	GitHub Actions	Automated backend and frontend build verification.

Version Note

The PDF brief recommends Java 21 and React 19. The local Mac environment currently has Java 17, so the implementation is Java 17-compatible while remaining Spring Boot 3.3 based. The CI file is prepared for Java 21 to align with the target production stack.

4. System Architecture

High-Level Architecture

The architecture uses a React frontend communicating with a Spring Boot backend through JWT-protected REST APIs. The backend organizes controllers, services, entities, repositories, security filters, and configuration classes. Flyway owns schema creation, and Docker Compose can run PostgreSQL, Redis, Prometheus, Grafana, backend, and frontend together.

```
React/Vite UI
  | JWT REST calls
Spring Boot Controllers
  | service orchestration
Auth, HR Workflow, Payroll, Insight, Notification, Audit Services
  | JPA repositories
PostgreSQL / H2 + Flyway schema
  | observability
Actuator metrics -> Prometheus -> Grafana
```

Important Backend Packages

Package	Responsibility
controller	REST API endpoints for authentication and HR workflows.
service	Business logic for auth, employees, leaves, payroll, insights, notifications, and audit.
model	JPA entities mapped to Flyway schema tables.
repository	Spring Data JPA repositories.
security	JWT generation, parsing, authentication filter, and user details service.
config	Security, CORS, database seeding, and application configuration.

5. Backend Implementation

Authentication and Authorization

NexusHR uses Spring Security with stateless JWT authentication. Demo users are seeded into the database with Argon2-hashed passwords. After login, the frontend receives a JWT containing the user role, and backend endpoints enforce access with role checks.

Endpoint	Purpose	Role
POST /api/auth/login	Login and receive JWT	Public
GET /api/dashboard/overview	Executive HR metrics	Authenticated
POST /api/employees	Create employee	ADMIN/MANAGER
DELETE /api/employees/{id}	Offboard employee	ADMIN
POST /api/leaves/{id}/decision	Approve/reject leave	ADMIN/MANAGER
POST /api/payroll	Generate payroll and payslips	ADMIN
GET /api/audit-logs	Sensitive audit trail	ADMIN

Database Modules

The database schema includes departments, roles, users, employees, attendance records, leave requests, payroll runs, payslips, performance reviews, notifications, and audit logs. Flyway migration V1 creates all core tables and constraints.

6. Frontend Implementation

The frontend is an operations console rather than a static landing page. Users sign in with seeded demo credentials, and the app stores the JWT in local storage. If a token expires or becomes invalid, the app clears the session and returns the user to login.

UI Area	Functionality
Login	Demo account login with JWT capture.
Metrics	Workforce size, open positions, attrition risk, payroll completion.
Payroll Action	Run payroll for a selected cycle from the admin console.
AI Insights	Attrition, attendance, and payroll recommendations.
Employee Table	Employee roster with department, status, and engagement score.
Leave Approvals	Approve or reject pending leave requests.
Attendance	Daily status and source of attendance events.
Notifications	Generated communication queue records.
Audit Logs	Sensitive operational action tracking.

Demo Credentials

Email	Password	Role
admin@nexushr.io	password123	ADMIN
manager@nexushr.io	password123	MANAGER
employee@nexushr.io	password123	EMPLOYEE

7. Execution Timeline

Phase	Days	Deliverables
Week 1	1-7	Project setup, Spring Boot API, security foundation, JPA schema, Flyway, employee and attendance bas
Week 2	8-14	React TypeScript frontend, JWT login, employee dashboard, leave workflow, payroll calculation, performa
Week 3	15-21	AI-style insight heuristics, notification records, audit logs, manager/admin console, role-aware UI.
Week 4	22-28	Docker Compose, Kubernetes manifests, GitHub Actions CI, Prometheus/Grafana, README, report PDF

Milestones

- Backend build passes with Maven.
- Frontend production build passes with Vite and TypeScript.
- Login returns JWT for seeded users.
- Protected endpoints respond with valid JWT.
- Payroll endpoint creates payroll runs and payslip records.
- Leave decision endpoint creates notification and audit records.
- Single command script starts frontend and backend together.

8. Security, Performance & Observability

Security Highlights

• JWT-based stateless API authentication.

• Argon2 password hashing for seeded demo accounts.

• Role-based endpoint protection through Spring Security and method security.

• CORS configuration limited to local frontend origins by default.

• Audit log records for sensitive operations such as payroll and employee changes.

• No API keys or secrets committed for paid/private third-party services.

Performance & Scalability

The API uses stateless authentication, relational indexes through unique constraints, container-ready deployment, Kubernetes HorizontalPodAutoscaler manifests, and separation between frontend and backend services. Redis is included for cache/session readiness in the Docker profile.

Observability

Spring Boot Actuator exposes health, metrics, and Prometheus endpoints. Docker Compose provisions Prometheus and Grafana so runtime metrics can be scraped and visualized during demos or production hardening.

9. Deployment & Operations

Mode	Command / File	Purpose
Local H2	./dev.sh or npm run dev	Starts backend and frontend with in-memory H2 database.
Docker Compose	docker compose up --build	Runs frontend, backend, PostgreSQL, Redis, Prometheus, Grafana.
Kubernetes	k8s/backend-deployment.yml	Backend deployment, service, readiness/liveness probes, HPA.
Kubernetes	k8s/frontend-deployment.yml	Frontend deployment and service.
CI	.github/workflows/ci.yml	Maven test and frontend build validation.
Monitoring	monitoring/prometheus/prometheus.yml	Scrapes backend Actuator Prometheus endpoint.

Local Run Command

```
./dev.sh
```

Docker Run Command

```
docker compose up --build
```

10. Visual Evidence & Demo Plan

The following screenshots should be captured after running the app locally or from the public deployment URL. They can be inserted into a final presentation or demo video:

Screenshot	What to Capture
Login Screen	JWT login page with admin demo credentials.
Dashboard Metrics	Workforce size, attrition risk, payroll completion, open positions.
AI Insights	Attrition, skill gap, attendance, and payroll recommendations.
Employee Roster	Employee table with department, status, and engagement score.
Leave Workflow	Approve/reject buttons and updated status.
Payroll Run	Run payroll action and new payroll history entry.
Audit Logs	Action entries after payroll and leave decisions.
Prometheus/Grafana	Metrics endpoint and monitoring dashboard.

Demo Video Flow

• Start app using ./dev.sh.

• Login as admin.

• Show dashboard metrics and AI insights.

• Approve one leave request.

• Run payroll for a new cycle.

• Open notifications and audit logs.

• Show backend health endpoint and repository structure.

11. Challenges, Learnings & Future Roadmap

Challenges Solved

• Moved from static demo data to real JPA-backed workflow data.

• Resolved local development friction with a single ./dev.sh runner.

• Handled stale JWT browser sessions by clearing invalid tokens in the frontend.

• Kept local development simple with H2 while preserving PostgreSQL-ready production deployment.

Key Learnings

The project demonstrates how enterprise systems require more than UI screens: authentication, authorization, database schema evolution, auditability, operations, deployment, and documentation all need to move together for a credible production-grade submission.

Future Roadmap

• Replace heuristic AI with Spring AI connected to OpenAI or Hugging Face.

• Add refresh tokens, logout token revocation, and stricter password policy.

• Add PDF payslip export and Excel workforce report export.

• Add real email/SMS provider integration.

• Deploy publicly on Render/Railway/AWS and record a 3-7 minute demo video.

• Run OWASP ZAP, Lighthouse, and load testing, then add results to this report.